

Latent Diffusion Models and Their Implementation in TorchDiff

Loghman Samani
samaniloqman91@gmail.com
Developer of TorchDiff

July 2, 2025

Abstract

This article introduces latent diffusion models (LDMs), a powerful approach to enhance the efficiency of conventional diffusion models by operating in a compressed latent space. We explore how LDMs, proposed by (1), address the computational challenges of traditional models through a two-stage process: perceptual compression using a variational autoencoder (VAE) and semantic compression via a diffusion process. Implemented in the PyTorch library TorchDiff, we detail the training of a VAE to map high-dimensional data (e.g., 256×256 RGB images) into a lower-dimensional latent space, followed by training a conditional DDIM model for efficient sampling. The article highlights the benefits of reduced computational costs and improved scalability, supported by a cross-attention mechanism for conditional generation with text prompts. Practical examples using TorchDiff (3) demonstrate the training and sampling processes, with results showcasing high-quality image generation. This work aims to make diffusion models more accessible and sustainable for real-world applications.

Keywords: Diffusion models, TorchDiff, LDMs, latent space, variational autoencoders, generative modeling, deep learning

Contents

1	Introduction	3
1.1	Contributions	3
2	Background and Related Work	3
2.1	Diffusion Models	3
2.2	Variational Autoencoders	4
3	Latent Diffusion Models: Concept and Architecture	4
3.1	Core Concept	4
3.2	Two-Stage Architecture	5
3.2.1	Stage 1: Perceptual Compression	5
3.2.2	Stage 2: Latent Diffusion	6
4	Methodology	6
4.1	Variational Autoencoder Design	6
4.1.1	Encoder Architecture	7
4.1.2	Latent Space Regularization	7
4.1.3	Decoder Architecture	7
4.1.4	Complete VAE Objective	7
4.2	Conditional Generation with Cross-Attention	8
4.2.1	Cross-Attention Mechanism	8
4.2.2	Conditional Training Objective	8
5	Implementation in TorchDiff	8
5.1	VAE Training Implementation	8
5.2	Conditional LDM Training	10
5.3	Sampling and Generation	12
6	Conclusion	14

1 Introduction

Diffusion models represent a class of likelihood-based generative models designed to capture every detail of the data distribution, including subtle and often imperceptible features to humans. This **mode-covering** behavior makes these models computationally intensive and challenging to train. The generation process typically requires multiple denoising steps, resulting in slow sampling procedures that significantly impact practical applications.

These computational challenges, combined with the energy consumption and carbon footprint associated with training and inference, have motivated researchers to develop more efficient approaches. The method we explore in this article—operating in a compressed latent space during both training and sampling—represents a significant advancement in making diffusion models more practical and sustainable.

Latent Diffusion Models (LDMs), as proposed by (1), address these limitations by decomposing the generation process into two stages:

1. **Perceptual compression:** Removing fine details while preserving overall structure
2. **Semantic compression:** Learning high-level patterns and features in the compressed space

This decomposition raises fundamental questions that we address throughout this article: How much compression can we apply while maintaining generation quality? What are the optimal compression strategies? How can we effectively implement these models in practice?

1.1 Contributions

This work makes the following contributions:

- A comprehensive explanation of LDM architecture and training procedures
- Detailed implementation guidelines using the TorchDiff library
- Mathematical formalization of the two-stage training process
- Practical code examples for both VAE and diffusion model training
- Analysis of computational benefits and quality trade-offs

2 Background and Related Work

2.1 Diffusion Models

Diffusion models learn to reverse a gradual noising process applied to data. Given a data distribution $p(\mathbf{x})$, the forward process gradually adds Gaussian noise:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}) \quad (1)$$

where β_t is a noise schedule. The reverse process learns to denoise:

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \Sigma_\theta(\mathbf{x}_t, t)) \quad (2)$$

2.2 Variational Autoencoders

Variational Autoencoders (2) learn to encode data into a latent space and reconstruct it with minimal loss. The VAE optimization objective combines reconstruction and regularization terms:

$$\mathcal{L}_{\text{VAE}} = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x}|\mathbf{z})] - \text{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z})) \quad (3)$$

3 Latent Diffusion Models: Concept and Architecture

3.1 Core Concept

Latent diffusion models operate in a compressed **latent space** rather than directly with raw data, leading to substantially improved efficiency compared to traditional pixel-space diffusion models. The key insight is that much of the computational cost in conventional diffusion models stems from processing high-dimensional data that contains redundant information.

Fig. 1 illustrates the fundamental trade-off between compression efficiency and reconstruction quality. This analysis guides our choice of compression strategy and helps determine optimal operating points for different applications.

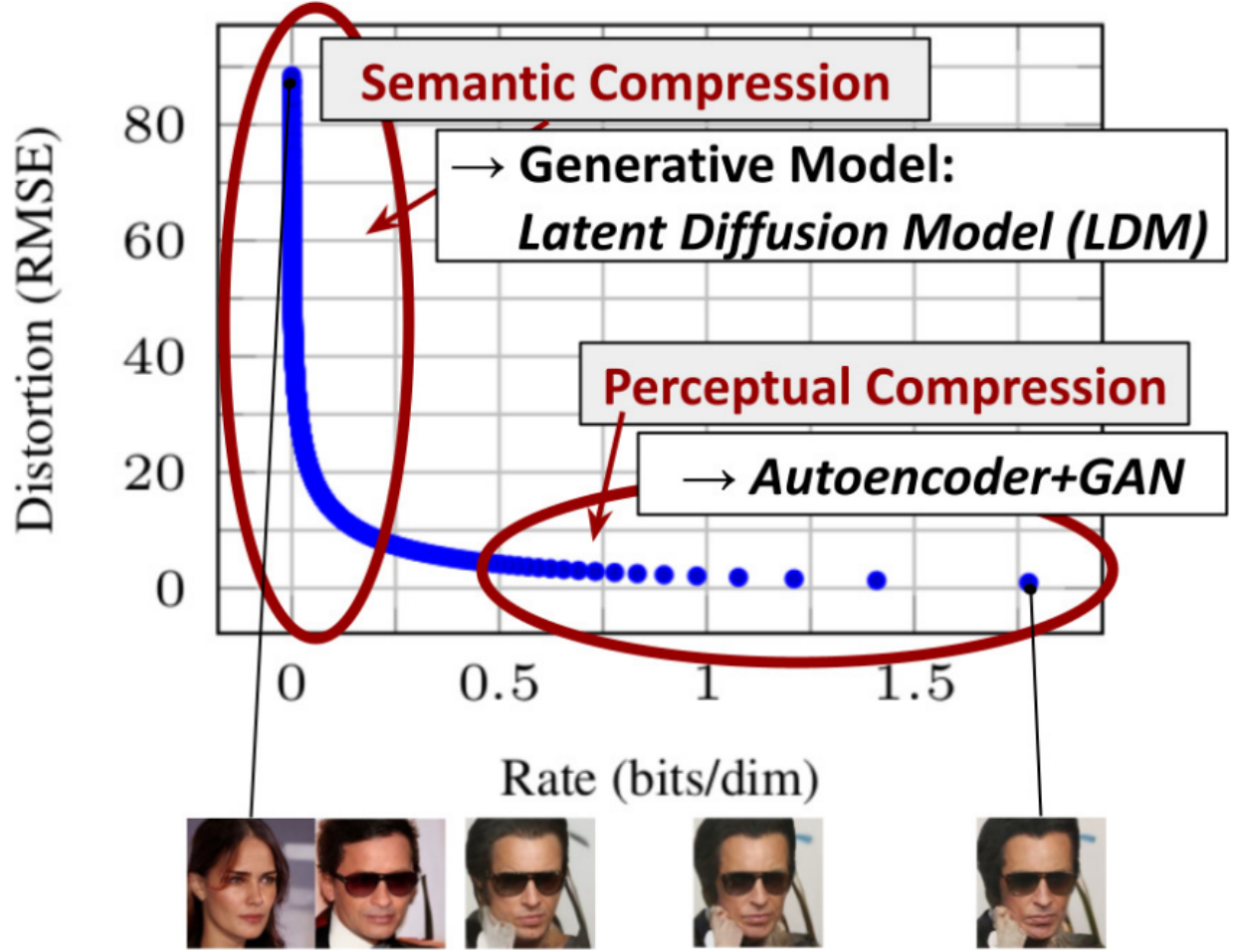


Figure 1: Comparison of semantic and perceptual compression techniques. The graph illustrates the trade-off between compression rate (bits/dim) and distortion (RMSE). Semantic compression (left) achieves higher compression rates but with increased distortion, while perceptual compression (right) maintains visual quality at moderate compression levels. The sample images demonstrate quality degradation with increasing compression levels.

3.2 Two-Stage Architecture

The LDM architecture consists of two main components that are trained sequentially:

3.2.1 Stage 1: Perceptual Compression

A variational autoencoder transforms high-dimensional input data $\mathbf{x} \in \mathbb{R}^{C \times H \times W}$ into a lower-dimensional latent representation $\mathbf{z} \in \mathbb{R}^{C_z \times H_z \times W_z}$, where:

$$\begin{aligned} H_z &= H/f \\ W_z &= W/f \\ C_z &\ll C \cdot f^2 \end{aligned} \tag{4}$$

with f representing the spatial downsampling factor.

3.2.2 Stage 2: Latent Diffusion

A diffusion model operates in the compressed latent space, learning the denoising objective:

$$\mathcal{L}_{\text{LDM}} = \mathbb{E}_{\mathbf{z}, \epsilon \sim \mathcal{N}(0, \mathbf{I}), t} [\|\epsilon - \epsilon_{\theta}(\mathbf{z}_t, t)\|_2^2] \quad (5)$$

where $\mathbf{z}_t$ represents the noisy latent at timestep t .

Fig. 2 offers a schematic overview of the LDM architecture, highlighting three main components: on the left (red background), the VAE maps input data into the latent space and reconstructs it back to the original data using the encoder and decoder, respectively; in the middle (green background), the main diffusion model operates in the latent space, which can be a DDPM, DDIM, or one of the SDEs available in TorchDiff (though theoretically, any diffusion model could be used); and on the right (light-gray background), conditionals are shown, which can be used to train a conditional LDM.

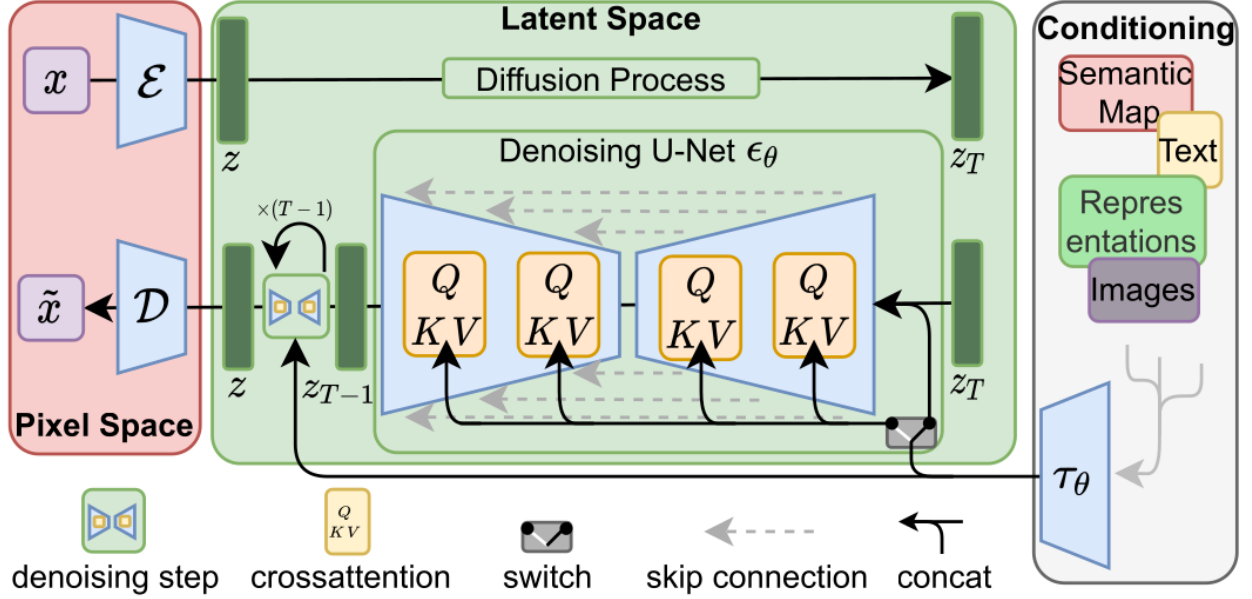


Figure 2: Schematic overview of the latent diffusion model architecture. The framework consists of three main components: (left, red) the VAE for perceptual compression and reconstruction, (center, green) the diffusion model operating in latent space, and (right, gray) the conditioning mechanism for controllable generation.

4 Methodology

4.1 Variational Autoencoder Design

The VAE component, implemented as `AutoencoderLDM` in TorchDiff, employs a U-Net-like convolutional architecture with the following key components:

4.1.1 Encoder Architecture

The encoder $E_\phi : \mathbb{R}^{C \times H \times W} \rightarrow \mathbb{R}^{C_z \times H_z \times W_z}$ uses progressive downsampling:

$$\mathbf{z} = E_\phi(\mathbf{x}) \quad (6)$$

4.1.2 Latent Space Regularization

The model supports two regularization approaches:

KL-Divergence Regularization For continuous latent representations, the encoder outputs parameters μ and $\log \sigma^2$, with latent sampling via the reparameterization trick:

$$\mathbf{z}_{\text{sample}} = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I}), \quad \sigma = \exp(0.5 \cdot \log \sigma^2) \quad (7)$$

The KL-divergence regularization term is:

$$\mathcal{L}_{\text{KL}} = -\frac{1}{2B \cdot S} \sum_{i=1}^{B \cdot S} \left(1 + \log \sigma_i^2 - \mu_i^2 - \sigma_i^2 \right) \cdot \beta \quad (8)$$

where B is batch size, S is latent dimensionality, and β is the regularization weight.

Vector Quantization For discrete latent representations, a codebook $\mathbf{E} \in \mathbb{R}^{N \times C_z}$ quantizes encoder outputs:

$$\mathbf{q}_i = \operatorname{argmin}_{\mathbf{e}_j \in \mathbf{E}} \|\mathbf{h}_i - \mathbf{e}_j\|_2^2 \quad (9)$$

The VQ loss combines codebook and commitment terms:

$$\begin{aligned} \mathcal{L}_{\text{codebook}} &= \frac{1}{M} \sum_{i=1}^M \|\operatorname{sg}[\mathbf{h}_i] - \mathbf{q}_i\|_2^2 \\ \mathcal{L}_{\text{commitment}} &= \gamma \cdot \frac{1}{M} \sum_{i=1}^M \|\mathbf{h}_i - \operatorname{sg}[\mathbf{q}_i]\|_2^2 \end{aligned} \quad (10)$$

where $\operatorname{sg}[\cdot]$ denotes stop-gradient and γ is the commitment coefficient.

4.1.3 Decoder Architecture

The decoder $D_\psi : \mathbb{R}^{C_z \times H_z \times W_z} \rightarrow \mathbb{R}^{C \times H \times W}$ reconstructs data from latent representations:

$$\hat{\mathbf{x}} = D_\psi(\mathbf{z}) \quad (11)$$

4.1.4 Complete VAE Objective

The total VAE loss combines reconstruction and regularization:

$$\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{reg}} \quad (12)$$

where $\mathcal{L}_{\text{recon}} = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2$ and $\mathcal{L}_{\text{reg}}$ is either $\mathcal{L}_{\text{KL}}$ or $\mathcal{L}_{\text{codebook}} + \mathcal{L}_{\text{commitment}}$.

4.2 Conditional Generation with Cross-Attention

For conditional generation $p(\mathbf{z}|\mathbf{y})$, where $\mathbf{y}$ represents conditioning information (e.g., text prompts), we employ a cross-attention mechanism within the U-Net architecture.

4.2.1 Cross-Attention Mechanism

Given an intermediate U-Net representation $\phi_i(\mathbf{z}_t) \in \mathbb{R}^{N \times d_\epsilon}$ and conditional embedding $\tau_\theta(\mathbf{y}) \in \mathbb{R}^{M \times d_\tau}$, the attention operation is:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}}\right)\mathbf{V} \quad (13)$$

where:

$$\begin{aligned} \mathbf{Q} &= \mathbf{W}_Q^{(i)} \phi_i(\mathbf{z}_t) \\ \mathbf{K} &= \mathbf{W}_K^{(i)} \tau_\theta(\mathbf{y}) \\ \mathbf{V} &= \mathbf{W}_V^{(i)} \tau_\theta(\mathbf{y}) \end{aligned} \quad (14)$$

4.2.2 Conditional Training Objective

The conditional LDM is trained with:

$$\mathcal{L}_{\text{cLDM}} = \mathbb{E}_{\mathcal{E}(\mathbf{x}), \mathbf{y}, \epsilon \sim \mathcal{N}(0, \mathbf{I}), t} [\|\epsilon - \epsilon_\theta(\mathbf{z}_t, t, \tau_\theta(\mathbf{y}))\|_2^2] \quad (15)$$

5 Implementation in TorchDiff

5.1 VAE Training Implementation

```

1  from torchvision import datasets, transforms
2  from torch.utils.data import DataLoader
3
4  # Define preprocessing pipeline
5  transform = transforms.Compose([
6      transforms.Resize(224),                # Standardize
7      transforms.CenterCrop(224),            # Center crop to
8      transforms.ToTensor(),                 # Convert to
9      transforms.Normalize((0.5, 0.5, 0.5),  # Normalize to [-1,
10                                     1])
11                                     (0.5, 0.5, 0.5))
12 ])
13 # Load datasets

```

```

14 train_dataset = datasets.ImageFolder(
15     root='./imagenet/train',
16     transform=transform
17 )
18 val_dataset = datasets.ImageFolder(
19     root='./imagenet/val',
20     transform=transform
21 )
22
23 # Create data loaders
24 train_loader = DataLoader(
25     train_dataset,
26     batch_size=64,
27     shuffle=True,
28     num_workers=8,
29     pin_memory=True,
30     drop_last=True
31 )
32 val_loader = DataLoader(
33     val_dataset,
34     batch_size=64,
35     shuffle=False,
36     num_workers=8,
37     pin_memory=True
38 )

```

Listing 1: Dataset preparation for VAE training

```

1 import torch
2 from torchdiff.utils import Metrics
3 from torchdiff.ldm import AutoencoderLDM, TrainAE
4
5 # Initialize VAE with optimized hyperparameters
6 vae = AutoencoderLDM(
7     in_channels=3, # RGB input
8     down_channels=[64, 128, 256, 512], # Encoder channel
9     progression
10    up_channels=[512, 256, 128, 64], # Decoder channel
11    progression
12    out_channels=3, # RGB output
13    dropout_rate=0.2, # Regularization
14    num_heads=8, # Multi-head attention
15    num_groups=32, # Group normalization
16    num_layers_per_block=2,
17    total_down_sampling_factor=2,
18    latent_channels=2, # Latent channel
19    dimension
20    num_embeddings=8192, # VQ codebook size

```

```

18     use_vq=False,                                # Use KL-divergence
19     beta=1e-6                                    # KL weight
20 )
21
22 # Configure optimizer with weight decay
23 optimizer = torch.optim.AdamW(
24     vae.parameters(),
25     lr=2e-4,
26     betas=(0.9, 0.999),
27     weight_decay=1e-4
28 )
29
30 # Initialize comprehensive metrics
31 metrics = Metrics(
32     device="cuda",
33     fid=True,                                     # Fr chet Inception Distance
34     ssim=True,                                   # Structural Similarity Index
35     lpips=True,                                 # Perceptual similarity
36     psnr=True                                   # Peak Signal-to-Noise Ratio
37 )
38
39 # Setup training configuration
40 vae_trainer = TrainAE(
41     model=vae,
42     optimizer=optimizer,
43     data_loader=train_loader,
44     val_loader=val_loader,
45     max_epoch=500,
46     metrics_=metrics,
47     device="cuda",
48     save_path="models/vae_ldm.pth",
49     val_frequency=10,
50     save_frequency=50,
51     gradient_clip_norm=1.0
52 )
53
54 # Execute training
55 vae_trainer()

```

Listing 2: VAE model training with TorchDiff

5.2 Conditional LDM Training

```

1 import torch
2 import torch.nn as nn
3 from torchdiff.ddim import HyperParamsDDIM, ReverseDDIM,
    ForwardDDIM

```

```

4 from torchdiff.ldm import TrainLDM
5 from torchdiff.utils import TextEncoder, NoisePredictor, Metrics
6
7 # Initialize noise prediction U-Net
8 noise_predictor = NoisePredictor(
9     in_channels=4,                                # Latent channels
10     from VAE
11     down_channels=[128, 256, 512, 1024],          # Encoder progression
12     mid_channels=[1024, 1024, 1024],              # Middle layers
13     up_channels=[1024, 512, 256, 128],            # Decoder progression
14     time_embed_dim=512,                           # Time embedding
15     dimension
16     y_embed_dim=768,
17     num_down_blocks=2,
18     num_mid_blocks=2,
19     num_up_blocks=2
20 )
21
22 # Initialize text encoder for conditioning
23 text_encoder = TextEncoder(
24     use_pretrained_model=True,
25     model_name="bert-base-uncased",
26     vocabulary_size=30522,
27     num_layers=6,
28     input_dimension=768,
29     output_dimension=768,
30     num_heads=8,
31     context_length=77,
32     dropout_rate=0.1,
33     qkv_bias=False,
34     scaling_value=4,
35     epsilon=1e-5
36 )
37
38 # Configure DDIM hyperparameters
39 ddim_params = HyperParamsDDIM(
40     num_steps=1000,                                # Training timesteps
41     tau_num_steps=100,                              # Inference
42     timesteps
43     beta_start=1e-4,                                # Noise schedule
44     start
45     beta_end=2e-2,                                  # Noise schedule end
46     beta_method="linear"
47 )
48
49 # Initialize diffusion processes
50 forward_ddim = ForwardDDIM(ddim_params)

```

```

47 reverse_ddim = ReverseDDIM(ddim_params)
48
49 # Configure optimizer
50 optimizer = torch.optim.AdamW([
51     {'params': noise_predictor.parameters(), 'lr': 1e-4},
52     {'params': text_encoder.parameters(), 'lr': 5e-5}
53 ], weight_decay=1e-4)
54
55
56 # Setup LDM trainer
57 ldm_trainer = TrainLDM(
58     model="ddim",
59     forward_model=forward_ddim,
60     noise_predictor=noise_predictor,
61     hyper_params=ddim_params,
62     compressor_model=vae, # Pre-trained VAE
63     conditional_model=text_encoder,
64     reverse_diffusion=reverse_ddim,
65     metrics_=metrics,
66     optimizer=optimizer,
67     objective=nn.MSELoss(),
68     data_loader=train_loader,
69     val_loader=val_loader,
70     max_epoch=1000,
71     device="cuda",
72     store_path="models/ldm_conditional.pth",
73     val_frequency=20
74 )
75
76 # Start training
77 ldm_trainer()

```

Listing 3: Training a conditional DDIM-based LDM

5.3 Sampling and Generation

```

1 from torchdiff.ldm import SampleLDM
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 # Initialize sampler with trained models
6 sampler = SampleLDM(
7     model="ddim",
8     reverse_diffusion=reverse_ddim,
9     noise_predictor=noise_predictor,
10    compressor_model=vae,
11    image_shape=(224, 224),

```

```
12     conditional_model=text_encoder,
13     batch_size=4,
14     in_channels=3,
15     device="cuda"
16 )
17
18 # Define text prompts for generation
19 prompts = [
20     "a serene mountain landscape at sunset",
21     "a futuristic city with flying cars",
22     "a detailed portrait of a wise old wizard",
23     "a vibrant underwater coral reef scene"
24 ]
25
26 with torch.no_grad():
27     images = sampler(
28         conditions=prompts,
29         save_images=True,
30         save_path="ldm_generated"
31     )
32
33 # Visualize results
34 fig, axes = plt.subplots(2, 2, figsize=(12, 12))
35 for i, (img, prompt) in enumerate(zip(images, prompts)):
36     row, col = i // 2, i % 2
37     axes[row, col].imshow(img.permute(1, 2, 0).cpu().numpy())
38     axes[row, col].set_title(prompt, fontsize=10)
39     axes[row, col].axis('off')
40
41 plt.tight_layout()
42 plt.savefig('generated_samples.png', dpi=300, bbox_inches='tight')
43 plt.show()
```

Listing 4: Sampling from trained conditional LDM

Fig. 3 shows generated results from a conditional LDM, as presented in the original LDM paper.

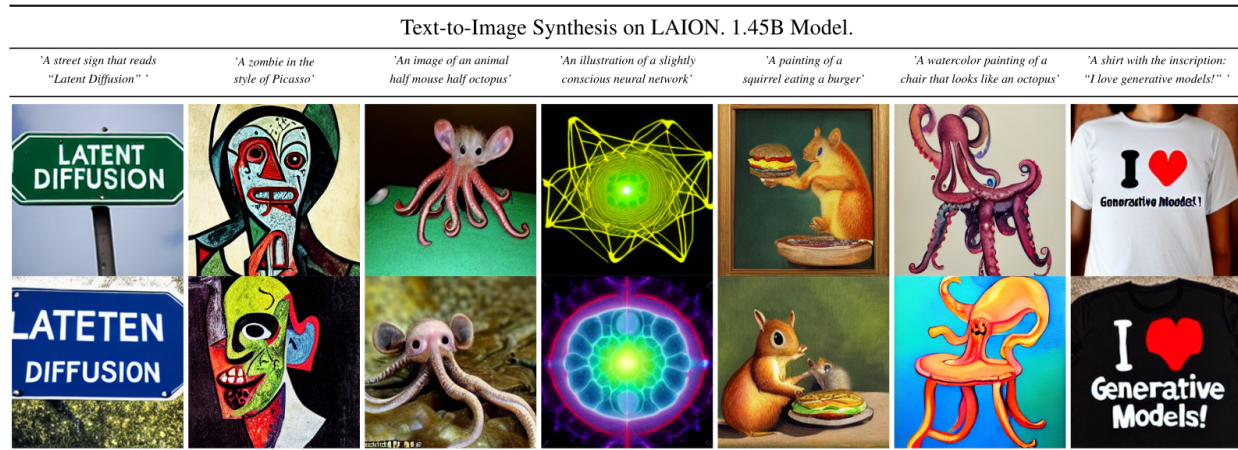


Figure 3: Examples of images generated by a trained conditional LDM, demonstrating the quality of output based on text prompts.

6 Conclusion

In this article, we have demonstrated how latent diffusion models (LDMs) offer a practical solution to the high computational demands of traditional diffusion models by leveraging a compressed latent space. We showed how to train and sample an LDM using TorchDiff library. The integration of a cross-attention mechanism enabled conditional generation, allowing the model to produce high-quality images based on text prompts, as evidenced by the generated results. By reducing dimensionality and focusing on semantic content, LDMs not only lower energy consumption but also enhance scalability for high-resolution images.

References

- [1] Rombach, Robin, et al. "High-resolution image synthesis with latent diffusion models." *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2022.
- [2] Kingma, Diederik P., and Max Welling. "Auto-encoding variational bayes." 20 Dec. 2013.
- [3] <https://github.com/LoqmanSamani/TorchDiff>.